

10. Grafi



Lezione 19 - @November 5, 2025

Grafi

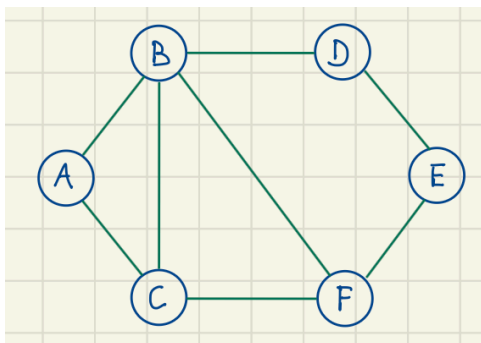
GRAFO $G(V, E)$

V insieme finito di NODI o VERTICI

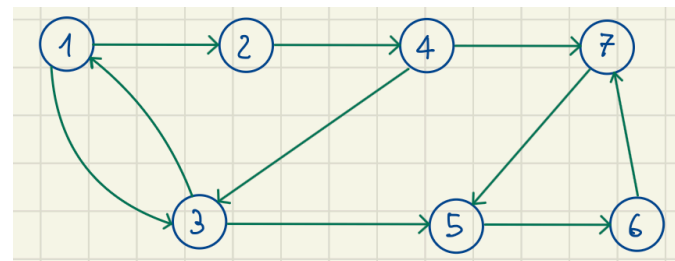
$E \subseteq V \times V$ insieme ARCHI (LATI, SPIGOLI)

Grafi:

- non orientati
- orientati/diretti



Non orientato



Orientato

Definizioni

$$\#V = n$$

$$\#E = m$$

- **incidenza**

Un arco (o spigolo) (x, y) si dice **incidente** sui suoi due vertici (nodi) di estremità, x e y .

- **adiacenza**

- **Grafi Non Orientati:** Se esiste un arco (x, y) , i vertici x e y sono **adiacenti**. La relazione è reciproca.
- **Grafi Orientati:** Se esiste un arco (x, y) che **esce da** x **ed entra in** y , si dice che y è **adiacente a** x . La relazione non è reciproca: x è adiacente a y solo se esiste anche un arco (y, x) .

- **vicini di** $v \in V$: vertici adiacenti a v

- nel grafo non orientato: i vicini di B sono A, C, D, F poiché tutti collegati fra loro.
- nel grafo non orientato: i vicini di 4 sono $7, 3$

- **Grado** $\delta(v)$ **di un vertice** v : numero di archi adiacenti su v

- $\delta_{in}(v)$ grado in ingresso
- $\delta_{out}(v)$ grado in uscita
- $\delta(v) = \delta_{in}(v) + \delta_{out}(v)$
- $\sum_v \delta_{in}(v) = m = \sum_v \delta_{out}(v)$

- **cammino** da x a y ($x, y \in V$):

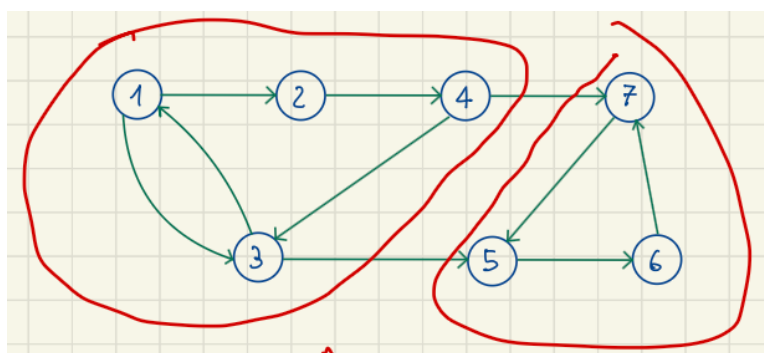
- sequenza di vertici $v_0, v_1, \dots, v_k \in V$ tc $v_0 = x$ $v_k = y$ e $(v_{i-1}, v_i) \in E$ per $i = 1, \dots, k$
- lunghezza del cammino = $\#archi(k)$
- cammino **semplice**: non contiene vertici ripetuti
- **y** è **raggiungibile** da **x** se $\exists$ cammino da **x** a **y**
- **ciclo**: cammino da un vertice v a v stesso
 - **ciclo semplice**: solo il vertice iniziale è ripetuto (alla fine)
- **catena** tra **x** e **y** ($x, y \in V$):
 - sequenza di vertici $v_0, v_1, \dots, v_k \in V$ tc $v_0 = x$ $v_k = y$ e $((v_{i-1}, v_i) \in E \text{ o } (v_i, v_{i-1}) \in E)$ per $i = 1, \dots, k$
 - **circuito** se $x = y$
- **grafo connesso**: tra ogni coppia di vertici esiste una catena

Grado fortemente connesso

tra ogni coppia di vertici esiste un cammino

- $G' = (V', E')$ è un sottografo di $G = (V, E)$ quando $V' \subseteq V$ e $E' \subseteq E \cap (V' \times V')$
- G' è il sottografo indotto da V' quando $E' = E \cap (V' \times V')$

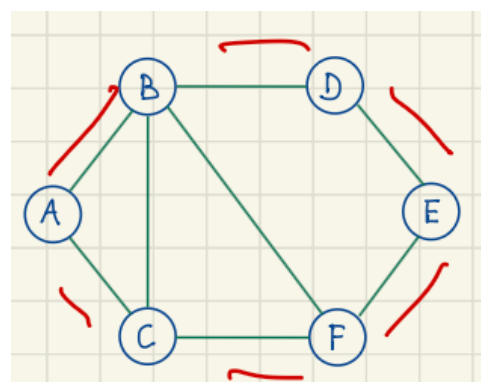
Una **componente fortemente connessa** è data dal sottografo indotto fortemente connesso massimale.



aree cerchiate: componenti fortemente connesse.

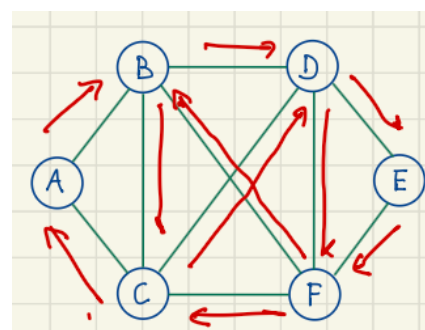
Circuito Hamiltoniano (grafi non orientati)

circuito che passa per ogni vertice del grafo una e una sola volta.

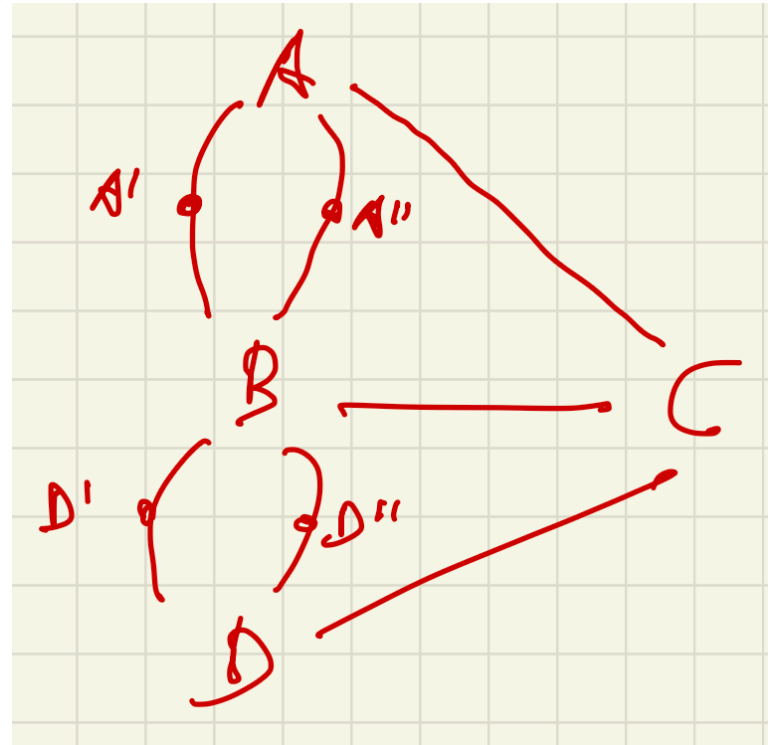
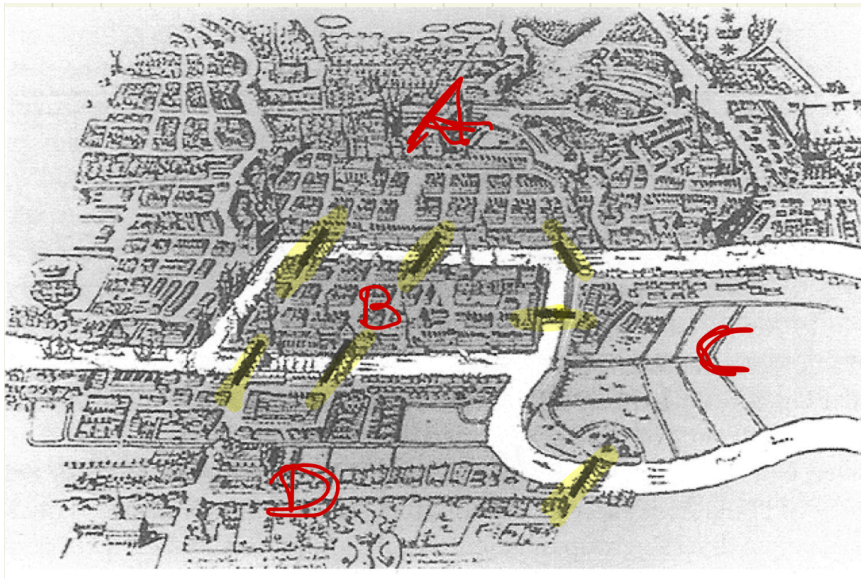


Circuito Euleriano

circuito che passa per ogni arco del grafo una e una sola volta



I ponti di KÖNIGSBERG



$\exists$ un circuito euleriano che attraversa tutti i ponti?

teorema: $\exists$ un circuito euleriano $\iff \forall v \quad \delta(v)$ è pari.



Lezione 20 - @November 7, 2025

Rappresentazione di Grafi

Lista di archi

voglio sapere il numero di vertici e rappresento solo gli archi. spesso è un array

$$G(V, E), n = \#V, m = \#E$$

Per i vertici $O(n)$, per gli archi $\Theta(m)$ - spazio totale.

Lista di adiacenza

array di liste

struttura principale $\rightarrow$ vertici

per ogni vertice $\rightarrow$ lista vertici adiacenti

lunghezza di ciascuna lista = grado vertice corrispondente

Per grafi non orientati

la lunghezza di ciascuna lista è pari al grado del vertice corrispondente

$$\sum \text{lunghezza delle liste} = 2m$$

$$\text{Spazio} = \Theta(n + m)$$

Per grafi orientati

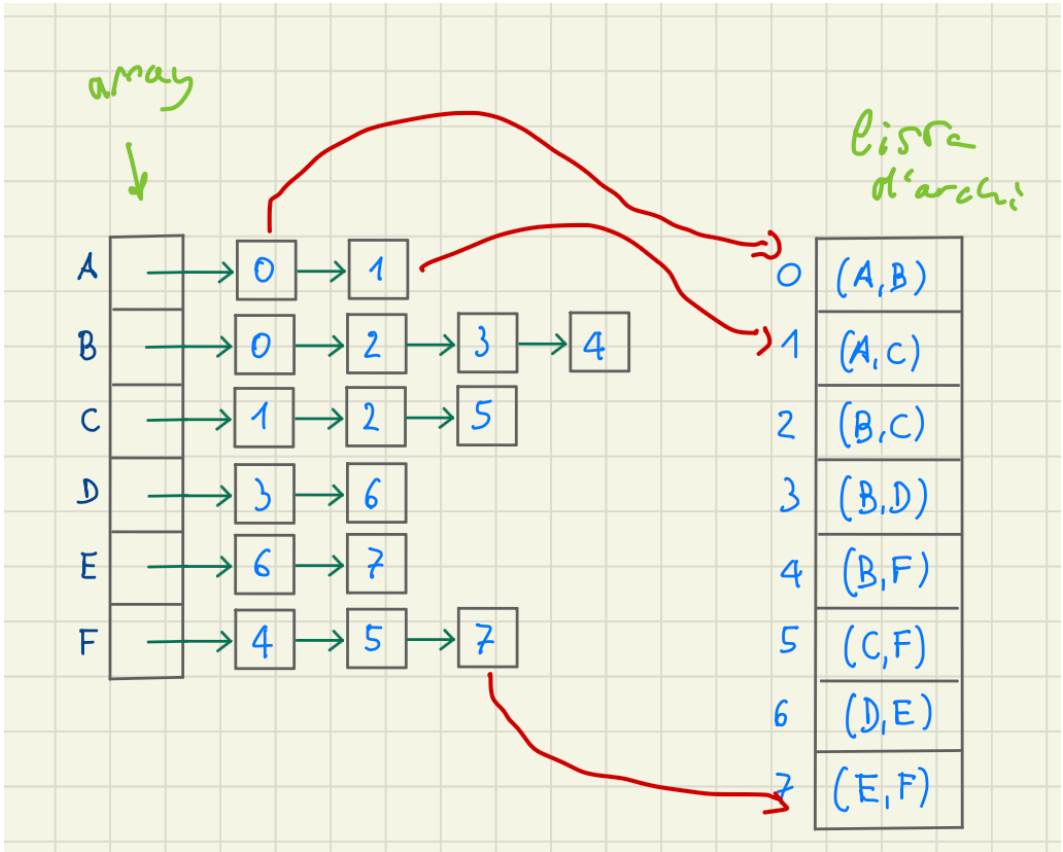
lunghezza lista associata al vertice $v = \delta_{out}(v)$

$$\sum \text{lunghezza delle liste} = \sum_v \delta_{out}(v) = m$$

Per i grafi orientati non è veloce cercare gli archi entranti

Lista di incidenza

spazio $\Theta(n + m)$

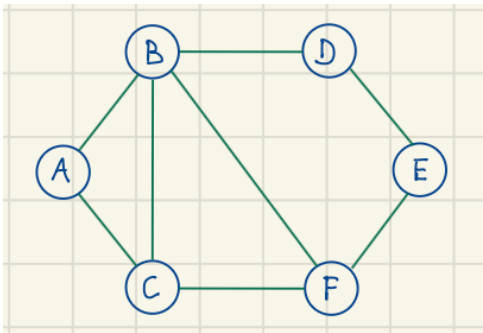


Matrice di adiacenza

- matrice **booleana** $n \times n$
- indici $\leftrightarrow$ vertici del grafo
- $M[u, v] = 1 \iff (u, v) \in E$

grafo non orientato $\rightarrow$ matrice simmetrica

spazio $\Theta(n^2)$



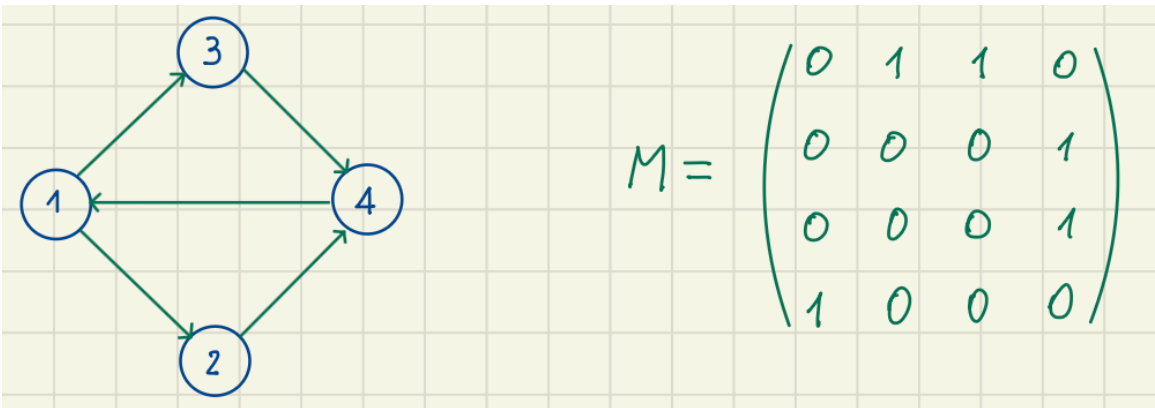
	A	B	C	D	E	F
A	0	1	1	0	0	0
B	1	0	1	1	0	1
C	1	1	0	0	0	1
D	0	1	0	0	1	0
E	0	0	0	1	0	1
F	0	1	1	0	1	0

$$\begin{pmatrix} 0 & 1 & 1 & 0 & 0 & 0 \\ 1 & 0 & 1 & 1 & 1 & 0 \\ 1 & 1 & 0 & 0 & 0 & 1 \\ 0 & 1 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 & 0 & 1 \\ 0 & 1 & 1 & 0 & 1 & 0 \end{pmatrix}$$

Per grafi orientati

- archi uscenti dal vertice **i** $\rightarrow$ riga **i**
- archi entranti nel vertice **i** $\rightarrow$ colonna **i**

Sia:



E sia M^2 l'operazione di moltiplicazione booleana tra matrici:

$$M^2 = \begin{pmatrix} 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 \end{pmatrix} \begin{pmatrix} 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 \end{pmatrix} = \begin{pmatrix} 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 \end{pmatrix}$$

- $M^2[i, j] = 1 \iff \exists$ cammino formato da 2 archi da i e j
- $M^k[i, j] = 1 \iff \exists$ cammino formato da k archi da i e j

Matrice di incidenza

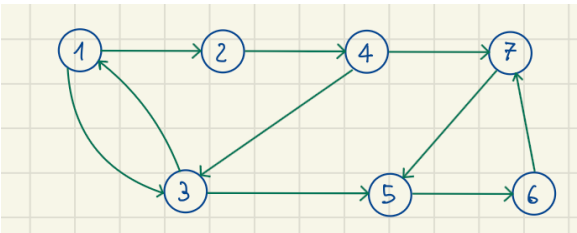
Per grafi non orientati

- righe $\rightarrow$ vertici
- colonne $\rightarrow$ archi
- 0/1
- spazio $O(n \cdot m)$

	(A,B)	(A,C)	(B,C)	(B,D)	(B,F)	(C,F)	(D,E)	(E,F)
A	1	1	0	0	0	0	0	0
B	1	0	1	1	1	0	0	0
C	0	1	1	0	0	1	0	0
D	0	0	0	1	0	0	1	0
E	0	0	0	0	0	0	1	1
F	0	0	0	0	1	1	0	1

Per grafi orientati

- 0
- -1 arco uscente
- 1 arco entrante



	(1,2)	(1,3)	(2,4)	(3,1)	(3,5)	(4,3)	(4,7)	(5,4)	(6,7)	(7,5)
1	1	1	0	-1	0	0	0	0	0	0
2	-1	0	1	0	0	0	0	0	0	0
3	0	-1	0	1	1	-1	0	0	0	0
4	0	0	-1	0	0	1	1	0	0	0
5	0	0	0	0	-1	0	0	1	0	-1
6	0	0	0	0	0	0	0	-1	1	0
7	0	0	0	0	0	0	-1	0	-1	1

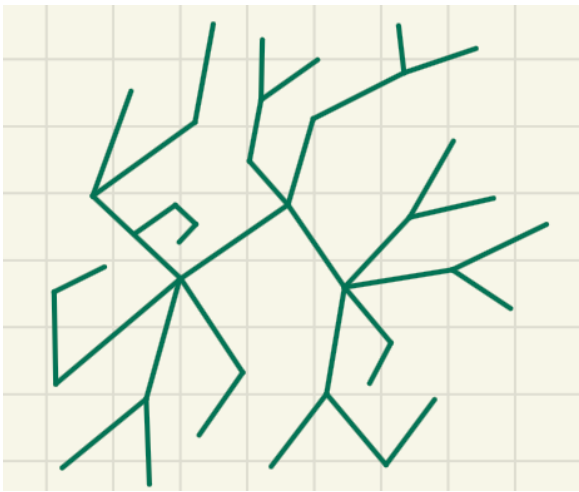
Alberi



Un albero è un grafo non orientato, connesso e privo di cicli

privo di cicli: tra ogni coppia di vertici $\exists!$ cammino

foresta: insieme di alberi



Prop 1 Sia $G(V, E)$ un albero $\Rightarrow \#E = \#V - 1$

dim $n = \#V$, induzione su n

$\boxed{n=1}$ • 1 vertice, 0 archi ✓

$\boxed{n>1}$ scelgo un vertice x qualunque

elimina x e i suoi archi incidenti da G

Siano $G_1 = (V_1, E_1) \dots G_k = (V_k, E_k)$ gli alberi ottenuti

$\#V_1, \dots, \#V_k < n \rightarrow$ ip. induzione

$\#E_1 = \#V_1 - 1, \#E_2 = \#V_2 - 1, \dots, \#E_k = \#V_k - 1$

$\#E = \#E_1 + \dots + \#E_k + k$ archi incidenti a x
 $= \#V_1 - 1 + \dots + \#V_k - 1 + k \Rightarrow \#V_1 + \#V_2 + \dots + \#V_k = \#V - 1$ vertice x

Prop 2 Sia $G=(V, E)$ non orientato e connesso

Se $\#E = \#V - 1$ allora G è un albero

Dim per assurdo: G non è un albero $\Rightarrow G$ contiene un ciclo

• Elimino un arco del ciclo e ripeto finché non ottengo un albero



$\rightarrow$ il grafo risultante:

connesso
privo di cicli con meno di $\#V - 1$ archi.

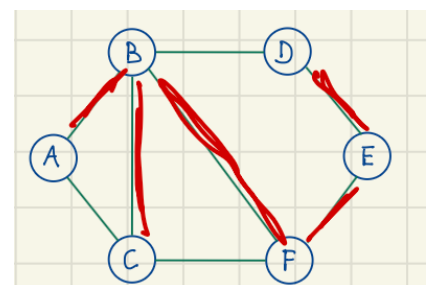
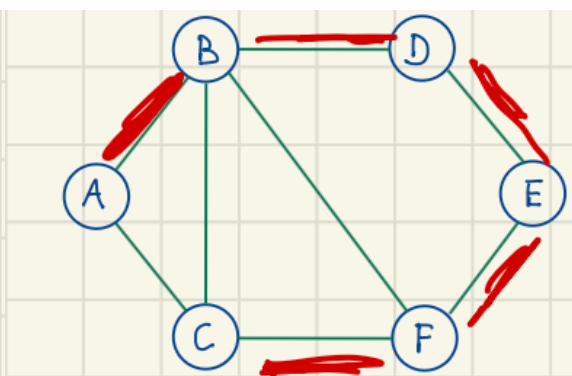
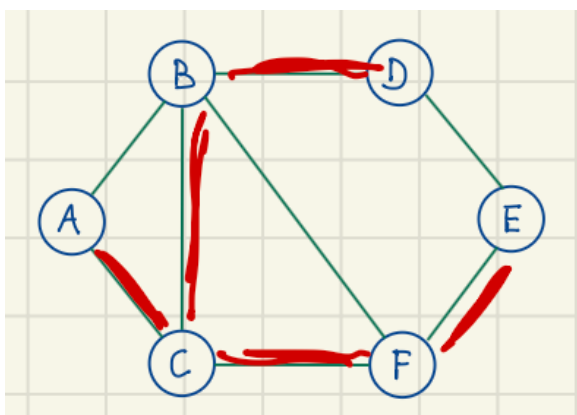
ASSURDO

se elimino archi partendo da $\#V - 1$ non posso trovare ancora $\#V - 1$ archi

Teo (Prop 1, 2) Un Grafo $G=(V, E)$ non orientato e connesso è un albero sse $\#E = \#V - 1$

Alberi ricoprenti - spanning tree

Dato $G = (V, E)$ grafo non orientato connesso, un albero ricoprente di G è un albero $G' = (V', E')$ con $V = V'$ e $E' \subseteq E$



Attraversamento di grafi

breadth first search

1. Si inizia visitando un vertice S

2. Si visitano i vertici adiacenti a S
3. Si visitano i vertici adiacenti ai vertici adiacenti a S , non ancora visitati
4. ...

Pseudo-codice

```
ALGORITMO visitaInAmpiezza(grafo  $G = (V, E)$ , vertice  $S$ )  $\rightarrow$  Albero
   $C \leftarrow$  coda vuota
   $T \leftarrow$  albero formato da solo  $S$ 
  marca  $S$  come raggiunto
   $C.enqueue(S)$ 
  WHILE NOT  $C.isEmpty()$  DO
     $u \leftarrow C.dequeue()$ 
    FOREACH  $(u, v) \in E$  DO
      IF  $v$  non è marcato come raggiunto THEN
        aggiungi  $v$  e  $(u, v)$  a  $T$ 
        marca  $v$  come raggiunto
         $C.enqueue(v)$ 
  RETURN  $T$ 
```



Lezione 21 - @November 10, 2025

Visita in ampiezza: tempi

Con lista di archi

- **WHILE** $\rightarrow n$ iterazioni
- **FOREACH** $\rightarrow m$ iterazioni ogni volta

$\rightarrow$ Tempo $O(n \cdot m)$

Con lista di adiacenza

L'algoritmo è lo stesso, ma il ciclo interno **FOR EACH** (u, v) scorre solo i veri vicini del nodo corrente. In totale, questo ciclo interno viene eseguito $2m$ volte in tutto il processo (ogni arco viene considerato due volte in un grafo non orientato).

- **WHILE** $\rightarrow n$ iterazioni
- **FOREACH** $\rightarrow \delta(u)$ iterazioni

$\rightarrow$ Tempo $O(n + m)$

Con matrice di adiacenza

Per trovare i vicini di un nodo estratto dalla coda, l'algoritmo deve scorrere l'intera riga della matrice corrispondente a quel nodo, che ha lunghezza n .

- **WHILE** $\rightarrow n$ iterazioni
- **FOREACH** $\rightarrow n$ passi a ogni iterazioni

$\rightarrow$ Tempo $O(n^2)$

Con matrice di incidenza

Per trovare i vicini di un nodo, bisogna scorrere la sua riga di lunghezza m (numero totale di archi) per vedere quali archi sono incidenti su di esso.

- **WHILE** $\rightarrow n$ iterazioni
- **FOREACH** $\rightarrow m$ passi a ogni iterazioni

$\rightarrow$ Tempo $O(n \cdot m)$

Visita in profondità - depth first search

1. Si inizia visitando un vertice S
2. partendo da S si percorre un cammino di vertici non ancora raggiunti visitandoli tutti, terminando su un vertice privo di vicini non ancora raggiunti
3. Si ripete dal passo precedente partendo dall'ultimo vertice sul cammino precedente che ha ancora un vertice non raggiunto

ALGORITMO visitaProfondità(grafo $G = (V, E)$, vertice S) $\rightarrow$ Albero

$T \leftarrow$ albero formato solo da S

visitaRicorsiva(G, S, T)

RETURN T

PROCEDURA visitaRicorsiva(grafo $G = (V, E)$, vertice u , albero T)

marca u come visitato

FOREACH $(u, v) \in E$ **DO**

IF v non è marcato come visitato **THEN**

 aggiungi v e (u, v) a T

visitaRicorsiva(G, v, T)